

AEGIS SECURE CONVERGENCE SYSTEMS LLC

WORKING PAPER — v0.2.3

Ethereal Cryptography: Formal Definitions and Security Model

Introducing

***SPECTER** — Stateless Protocol for Ephemeral
Cryptographic Token Endorsement and Revocation*

Quantum Forward Secrecy
Ephemeral Assembly Security
Full Ethereal Security

Russell Aydelotte

Aegis Secure Convergence Systems LLC

Osceola, Indiana · May 2026

v0.2.3 · Subject to revision

© 2026 Russell Aydelotte, Aegis Secure Convergence Systems LLC. Licensed under CC BY-NC 4.0.

Abstract

We introduce **Ethereal Cryptography**, a cryptographic paradigm in which private key material is never persistently stored — not before, during, or after use. An *Ethereal Key* is assembled from threshold shares at the moment of use, performs exactly one cryptographic operation, and is provably destroyed within a formally bounded threat model. The resulting tokens are epoch-scoped, binding their validity to a temporal context that expires before any known quantum attack can complete.

We show that this paradigm provides **Quantum Forward Secrecy (QFS)** — a security property that is strictly stronger than classical forward secrecy when epoch lifetimes are negligible relative to the quantum attack horizon, and independent of the mathematical hardness of the underlying signature scheme within that constraint. When composed with a post-quantum signature algorithm (e.g., ML-DSA-87), the construction achieves **Full Ethereal Security**: token forgery is computationally infeasible for both classical and quantum adversaries within our threat model, and key-material theft is architecturally precluded.

We present **SPECTER** (Stateless Protocol for Ephemeral Cryptographic Token Endorsement and Revocation) as the first formally specified instantiation of an Ethereal Key Scheme, and identify open problems required for a complete formal treatment.

Implementation status. This paper specifies the Ethereal Key Scheme and SPECTER as a *design* and proves the security properties of that design. The currently deployed SPECTER build conforms to this specification with three documented exceptions — fresh per-ceremony key generation (rather than deterministic epoch-keyed derivation), single-host simulation of distributed share custody, and a cryptographic threshold of $t = 2$ (with the 3-of-5 quorum enforced one layer up, at the governance voting gate). These are detailed in §9.4; none affects the QFS, EAS, or Epoch Isolation results.

1. Introduction

Every known attack on a public-key cryptographic system — classical or quantum — depends on a single precondition:

The private key must exist in a recoverable form at some point in time.

Classical adversaries steal or factor their way to the private key. Quantum adversaries (Shor's algorithm) recover the private key from the public key in polynomial time — but only if the private key is *still useful* when the quantum computation completes.

The entire discipline of Post-Quantum Cryptography (PQC) (Bernstein & Lange 2017) addresses this by replacing mathematically-weak primitives (ECDLP, integer factoring) with mathematically-strong ones (lattice problems, hash-based signatures). This is necessary and correct. But it addresses the *algorithm* layer, not the *architecture* layer.

Ethereal Cryptography addresses the architecture layer.

If private key material never persists — assembled from threshold shares only during an authorized ceremony, used once, and destroyed within a bounded threat model — then even a computationally unbounded quantum adversary arriving after the ceremony finds nothing to attack. The key is gone. The epoch is expired.

Key Insight

Traditional cryptography assumes that security depends on the computational hardness of recovering a persistent private key. Ethereum Cryptography rejects this assumption.

Instead, it enforces a stronger invariant:

A private key that does not persist cannot be stolen, and a key that no longer exists cannot be attacked.

We qualify this claim carefully: QFS is conditional on the engineering invariant that epoch lifetimes T_e are negligible compared to the quantum attack horizon $T_q(\lambda)$. This is an empirical deployment assumption, not a mathematical theorem. We state it explicitly wherever the result depends on it.

1.1 Our Contributions

1. **Formal definition of an Ethereum Key Scheme (EKS)** — a tuple of algorithms satisfying Ephemeral Assembly, Single-Use Destruction, and Epoch Binding.
2. **A threat model for secure memory erasure** — explicitly bounding the adversary capabilities under which Destroy provides meaningful security, addressing the central load-bearing assumption.
3. **Formal definition of Quantum Forward Secrecy (QFS)** — a game-based security property capturing resistance to post-ceremony quantum attack, with a proof reducing to PRF security of HKDF and the UF-CMA property of the signing primitive.
4. **The Ephemeral-Quantum Gap Theorem** — a conditional theorem: if epoch lifetime T_e is negligible compared to $T_q(\lambda)$, then any EKS satisfying our threat model provides QFS.
5. **Full Ethereum Security (FES)** — the composed security property under a PQC signing algorithm.
6. **Liveness properties** — ceremony failure model and availability guarantees, complementing the safety properties.
7. **Multi-Ceremony Composition Conjecture** — with a reduction sketch to PRF security, promoting this from an open problem to a supported claim.
8. **SPECTER** — a concrete EKS construction with a deployed implementation, formally analyzed against the EKS definition.

1.2 Time as a Security Primitive

Traditional cryptography treats time as an external parameter. Keys exist indefinitely; adversaries race to break them before they are rotated. Security is a statement about how long a key *resists* attack.

Ethereal Cryptography internalizes time into the security model itself.

The validity of a key is not indefinite — it is explicitly bounded, and every protocol operation enforces that bound. Assemble produces a key valid for exactly one epoch. Sign consumes that key in one operation. Destroy closes the window. Verify rejects anything outside the window.

Security is therefore not a function of how long a key can resist attack. It is a function of whether the key remains valid long enough for an attack to matter. When epoch lifetimes are short relative to the quantum attack horizon, they cannot. The attack succeeds — and arrives to find an empty room.

This reframing has a precise consequence: the relevant security parameter is not the bit-length of the key but the ratio $T_e / T_q(\lambda)$. A 256-bit key with $T_e = 10$ years provides weaker temporal security than a 128-bit key with $T_e = 1$

hour, even if the 256-bit key is classically stronger. Ethereum Cryptography makes this trade-off explicit and exploitable by design.

1.3 Architectural vs. Mathematical Security

Two complementary approaches exist for resisting cryptographic attack:

Post-Quantum Cryptography replaces mathematically-weak primitives with mathematically-strong ones. It makes key recovery *harder* — requiring resources no known quantum computer possesses. Security depends on the intractability of the underlying hard problem.

Ethereal Cryptography eliminates the value of key recovery entirely. Even a computationally unbounded adversary who recovers the private key gains nothing if the key is already expired. Security does not depend on computational hardness at all — it depends on a temporal invariant.

Post-quantum cryptography strengthens the difficulty of key recovery. Ethereum Cryptography eliminates the utility of key recovery. These approaches are orthogonal and composable.

Full Ethereum Security (§7) is achieved by composing both: the signing primitive resists quantum attack, and the architecture ensures that even a successful attack yields nothing useful.

2. Preliminaries

2.1 Notation

Symbol	Meaning
λ	Security parameter
PPT	Probabilistic polynomial-time
QPT	Quantum polynomial-time (BQP)
$\text{negl}(\lambda)$	Negligible function in λ
$\Pr[E]$	Probability of event E
A^O	Algorithm A with oracle access to O
$\leftarrow$	Uniform random assignment
$:=$	Definition
t_{now}	Current wall-clock time (distinct from threshold parameter t)
t	Threshold parameter in (t,n)-secret sharing
T_e	Epoch lifetime (wall-clock duration)
$T_q(\lambda)$	Quantum attack horizon for security parameter λ
T_e/T_q	Temporal security ratio — the key parameter of Ethereum Cryptography

Notation discipline: Throughout this paper, lowercase t denotes the threshold parameter in a (t,n) -secret sharing scheme. Wall-clock time is written t_now to avoid ambiguity. Epoch validity is written $\text{EpochValid}(\text{eid}, t_now)$.

2.2 Threshold Secret Sharing

A (t, n) -threshold secret sharing scheme (Shamir 1979) is a tuple (Share, Reconstruct) such that:

- **Share**(s, t, n) $\rightarrow (\sigma_1, \dots, \sigma_n)$: splits secret s into n shares
- **Reconstruct**($\sigma_{\{i_1\}}, \dots, \sigma_{\{i_t\}}$) $\rightarrow s$: recovers s from any t shares
- **Privacy**: any subset of fewer than t shares is statistically independent of s

2.3 Epoch System

An **epoch system** is a tuple (EpochGen, EpochValid, EpochBind) where:

- **EpochGen**(λ) $\rightarrow (\text{eid}, T_start, T_end)$: generates epoch identifier and validity window
- **EpochValid**(eid, t_now) $\rightarrow \{0,1\}$: returns 1 iff current time $t_now \in [T_start, T_end]$
- **EpochBind**(eid, data) $\rightarrow \text{bytes}$: deterministically binds data to an epoch

Epochs are *non-overlapping* and *non-renewable*: once T_end passes, no new tokens can be issued for that epoch and no extension is possible.

Entropy requirement: The `aux` input to EpochBind MUST include fresh entropy or domain-separated context per epoch. Reuse of `aux` across distinct epochs without domain separation violates the Epoch Isolation invariant (Definition 5.3): if $\text{EpochBind}(\text{eid}_1, \text{aux}) = \text{EpochBind}(\text{eid}_2, \text{aux})$, then $\text{sk}_{\{e_1\}} = \text{sk}_{\{e_2\}}$, and a recovered key from one epoch directly applies to the other. Implementations MUST incorporate the epoch identifier as a mandatory component of `aux` (as SPECTER does via $\text{SHA3-256}(\text{eid} \parallel "||" \parallel \text{data})$).

Common Failure Modes in Epoch Binding

The following implementation errors collapse Epoch Isolation and MUST be treated as security-critical bugs:

Failure Mode	Consequence
<code>aux</code> reused across epochs without <code>eid</code>	$\text{EpochBind}(\text{eid}_1, \text{aux})$ may equal $\text{EpochBind}(\text{eid}_2, \text{aux})$; $\text{sk}_{\{e_1\}} = \text{sk}_{\{e_2\}}$ — epoch isolation collapses entirely
<code>eid</code> generated from low-entropy source	Epoch identifiers become predictable; adversary can anticipate EpochBind outputs
<code>eid</code> omitted from EpochBind salt	Two ceremonies with the same IKM produce identical keys regardless of <code>eid</code> value
<code>aux</code> generated before t shares are confirmed	Partial ceremony with <code>aux</code> fixed; abort+retry with same <code>aux</code> re-derives the same sk_e in a new epoch
EpochBind output truncated	Reduced domain separation; increased collision probability across epochs

Implementation rule: Generate `aux` fresh *inside* the ceremony window, after quorum confirmation, using a cryptographically secure random source. `eid` MUST be included as a mandatory prefix, not an optional component.

2.4 PRF Security

A function family $F = \{F_k : \{0,1\}^* \rightarrow \{0,1\}^\lambda\}$ is a **secure PRF** if no PPT distinguisher D can distinguish $F_k(\cdot)$ from a random oracle with non-negligible advantage. We model HKDF-SHA3-512 as a secure PRF in the random oracle model.

3. Threat Model and the Destroy Primitive

The Destroy operation is the load-bearing assumption of this paper. We address it explicitly rather than asserting it.

3.1 The Secure Erasure Problem

In practice, secure memory zeroing faces several implementation hazards:

- **Compiler optimization:** compilers may elide writes to memory that is about to go out of scope, treating zeroing as a dead write.
- **OS paging:** the kernel may swap pages containing key material to disk before Destroy is called.
- **CPU cache and register retention:** values may persist in L1/L2 cache lines or processor registers after memory is zeroed.
- **Language runtime semantics:** garbage-collected languages (Python, Java) do not guarantee when or whether objects are zeroed.
- **Hibernation and VM snapshots:** system state may be written to persistent storage at any point.

We do not require perfect physical erasure. We do not claim that key material is unrecoverable by every conceivable physical or microarchitectural means.

Instead, we require something more precisely bounded: within the Software Memory Adversary model, no adversary can recover *usable* key material after Destroy returns.

Destroy is therefore not a claim about physics. It is a protocol-enforced temporal boundary on key existence.

After Destroy, the key does not exist within the protocol's threat model — and protocol-level operations (Verify, Revoke, subsequent ceremonies) enforce that boundary. An adversary who circumvents Destroy via a physical attack is outside this model and requires hardware enclave mitigations (see §9.2).

Definition 3.1 — Software Memory Adversary (SMA)

A **Software Memory Adversary** is a PPT (or QPT) algorithm A that:

- Has access to the running process's observable outputs (tokens, public keys, witness records, network traffic)
- Can read process memory only *before* the ceremony begins or *after* Destroy returns — but NOT during the assembly/sign/destroy window
- Has no physical access to hardware (no cold-boot attacks, no DMA attacks, no power analysis)
- Cannot induce or observe paging of the key material region during the ceremony window
- Cannot interrupt or suspend the ceremony thread

Out of scope for the SMA model (acknowledged explicitly):

- Physical memory attacks (cold-boot, DMA, hardware probing)
- Hypervisor-level memory inspection
- OS kernel compromise

- CPU microarchitectural attacks (Spectre/Meltdown variants on key material)
- Hibernation/snapshot attacks during the ceremony window

Systems requiring stronger guarantees against physical adversaries should supplement EKS with hardware security enclaves (SGX, TrustZone, Vetric HRoT).

Concrete mitigations within the SMA model: Implementations SHOULD apply the following OS/runtime controls to narrow the gap between the SMA model and physical reality:

- `mlock(2)` / `VirtualLock`: Pin key material pages to RAM, preventing the kernel from swapping them to disk during the ceremony window.
- `MADV_DONTDUMP` (Linux): Exclude key material regions from core dumps.
- `mlockall(MCL_CURRENT | MCL_FUTURE)`: Lock all current and future pages for the ceremony process.
- Real-time scheduling (`SCHED_FIFO` / `REALTIME_PRIORITY_CLASS`): Reduce preemption risk during the Sign→Destroy critical window.
- `madvise(MADV_WIPEONFORK)`: Prevent key material from being inherited by forked processes.

None of these eliminate physical memory attacks. They narrow the window of exposure within the SMA model and reduce implementation-layer risk. **The SMA model remains a protocol boundary, not a physics claim.** Enclave integration (§9.2) is the path to a stronger physical adversary model.

Definition 3.2 — Secure Destroy

Under the SMA model, **Destroy**(`sk_e`) satisfies *secure erasure* if:

1. All bytes of `sk_e` are overwritten with zeros using a memory barrier that prevents compiler reordering (e.g., `memset_s` in C, `explicit_bzero` in glibc/BSD (or `OPENSSL_cleanse` in OpenSSL); SPECTER's `SecureBuffer` zeroizes via `ctypes.memset`).
2. The zeroing occurs before the function returns to the caller.
3. The ceremony is executed atomically with respect to OS scheduling (no preemption between Sign and Destroy in the critical window).

Remark: Condition 3 is an implementation requirement, not a mathematical one. It can be enforced via OS scheduling primitives (e.g., `mlockall`, `MADV_DONTDUMP`, real-time scheduling priority). We treat it as a protocol requirement that implementations must satisfy, analogous to the requirement that a nonce is never reused in AES-GCM.

Remark on Python: CPython's `bytearray` and `ctypes` buffer types support explicit zeroing via `memset`. The standard `bytes` type does NOT guarantee zeroing. SPECTER's `SecureBuffer` uses `ctypes.create_string_buffer` with explicit `memset` calls to satisfy Definition 3.2 within the SMA model.

4. The Ethereal Key Scheme

Definition 4.1 — Ethereal Key Scheme (EKS)

An **Ethereal Key Scheme** is a tuple of algorithms:

$\text{EKS} = (\text{Setup}, \text{ShareGen}, \text{Assemble}, \text{Sign}, \text{Verify}, \text{Destroy}, \text{Revoke})$

satisfying the following:

Setup(λ) $\rightarrow$ pp Generates public parameters including epoch system, KDF parameters, and signing algorithm configuration.

ShareGen(pp, t, n) $\rightarrow$ (pk, σ_1 , ..., σ_n , aux) Generates n key shares for a (t,n)-threshold scheme. Returns a public key pk (to be published) and shares σ_i (distributed to share holders). The assembled private key sk is **never output**. aux contains entropy material for epoch binding.

Assemble($\sigma_{\{i_1\}}$, ..., $\sigma_{\{i_t\}}$, eid, aux) $\rightarrow$ (sk_e, pk_e, W) Given t shares and an epoch identifier eid, assembles:

- sk_e: ephemeral private key in a SecureBuffer (never written to disk)
- pk_e: ephemeral public key (published with tokens)
- W: witness record (quorum proof, not key material)

Protocol constraints:

- Assemble MUST verify EpochValid(eid, t_now) = 1 before proceeding; it MUST abort and call Destroy on any partial key material if the epoch is expired. This check is required for the EAS proof (§5.2).
- sk_e MUST be passed to Sign and then to Destroy within a single atomic ceremony window. It MUST NOT be serialized, logged, or returned to the caller.

Sign(sk_e, m, eid) $\rightarrow$ σ_{token} Signs message m under sk_e, binding the signature to epoch eid. Called exactly once per assembled key.

Verify(pk_e, m, σ_{token} , eid, t_now) $\rightarrow$ {0,1} Verifies signature validity, epoch validity at time t, and non-revocation of eid.

Destroy(sk_e) $\rightarrow$ $\perp$ Securely zeros all bytes of sk_e per Definition 3.2. Called unconditionally after Sign (even on error — in a finally block).

Revoke(eid, reason) $\rightarrow$ rev_record Permanently marks epoch eid as revoked.

Definition 4.2 — Correctness

An EKS is **correct** if for all λ , pp $\leftarrow$ Setup(λ), valid epochs eid (with EpochValid(eid, t_now) = 1), and messages m:

$$\Pr[\text{Verify}(\text{pk}_e, m, \text{Sign}(\text{sk}_e, m, \text{eid}), \text{eid}, t_{\text{now}}) = 1] = 1$$

where (pk_e, sk_e, W) $\leftarrow$ Assemble(t shares from ShareGen(pp, t, n), eid, aux).

4.1 Liveness Properties

A complete treatment requires both safety (bad things don't happen) and liveness (good things eventually happen). We define the following.

Definition 4.3 — Ceremony Liveness: An EKS satisfies *ceremony liveness* if: whenever at least t valid shares are presented and EpochValid(eid, t_now) = 1, Assemble completes and Sign produces a valid token within bounded time.

Definition 4.4 — Graceful Degradation: An EKS satisfies *graceful degradation* if: when fewer than t shares are presented, Assemble aborts cleanly — producing no partial key material, no partial token, and leaving no sk_e in memory. The epoch is not consumed (it may be retried within its validity window).

Definition 4.5 — Epoch Availability: An EKS satisfies *epoch availability* if: for any valid epoch eid with $T_{\text{end}} - T_{\text{start}} = T_e$, at least one ceremony can succeed within $[T_{\text{start}}, T_{\text{end}}]$ given a non-faulty quorum. (This is a protocol-level liveness property, not a cryptographic one — it depends on share holder availability.)

Remark on availability vs. security tradeoff: The threshold parameter t directly trades liveness for security. Lower t improves ceremony availability (fewer share holders needed) but weakens threshold privacy. A $t=3$, $n=5$ configuration tolerates two simultaneous share-holder failures while maintaining meaningful threshold security. (Note: SPECTER's deployed cryptographic threshold is $t=2$; the 3-of-5 quorum is enforced one layer up, at the LOCUST council *voting* gate that authorizes the ceremony — see §9.1 and §9.4.)

5. Security Properties

5.1 Standard Properties

Unforgeability (UF-CMA): No PPT adversary A , given pk_e and signing oracle access, can produce a valid (m, σ) for an unsigned m^* , except with probability $\text{negl}(\lambda)$. Reduces to UF-CMA of the underlying signature scheme.

Threshold Privacy: No adversary holding fewer than t shares learns anything about sk_e . Follows directly from Shamir (1979) statistical privacy.

5.2 Ephemeral Assembly Security (EAS)

Definition 5.1 — EAS Game:

Game $EAS_A(\lambda)$: $pp \leftarrow \text{Setup}(\lambda)$ $(pk, \sigma_1, \dots, \sigma_n, aux) \leftarrow \text{ShareGen}(pp, t, n)$ $eid \leftarrow \text{EpochGen}(\lambda)$ $(sk_e, pk_e, W) \leftarrow \text{Assemble}(\{t \text{ shares}\}, eid, aux)$ $\sigma_token \leftarrow \text{Sign}(sk_e, m, eid)$ $\text{Destroy}(sk_e)$ // sk_e zeroed here

// Adversary receives everything post-ceremony, // including ALL shares (worst case: all holders corrupt) $(m', \sigma') \leftarrow A(pp, pk, pk_e, \sigma_token, W, eid, m, \sigma_1, \dots, \sigma_n)$

// A wins by forging a signature on a fresh, never-signed message return 1 if $m' \neq m$ and $\text{Verify}(pk_e, m', \sigma', eid, t_now) = 1$

Definition 5.2 — EAS Security (under SMA model): An EKS satisfies EAS if for all PPT adversaries A operating within the Software Memory Adversary model:

$$\Pr[EAS_A(\lambda) = 1] \leq \text{negl}(\lambda)$$

Proof: Post-Destroy, A has: $(pk_e, \sigma_token, W, eid, \text{all shares})$. Can A recover sk_e ?

Reassembling the shares gives $\text{Reconstruct}(\sigma_{\{i_1\}}, \dots, \sigma_{\{i_t\}}) = \text{the original Shamir secret } s$. But sk_e was derived as:

$$sk_e = \text{KDF}(s, \text{salt} = \text{EpochBind}(eid, aux)) \text{ // } s = \text{Shamir secret || optional ML-KEM seed}$$

A fresh call to Assemble with the same shares but the same eid would produce the identical sk_e — but $\text{EpochValid}(eid, t_now) = 0$ once the epoch has expired. Assemble MUST reject expired epochs (this is a protocol requirement).

If A attempts to use the recovered shares for a *different* epoch e_2 : $sk_{\{e_2\}} = \text{KDF}(s, \text{salt} = \text{EpochBind}(e_2, aux'))$ is computationally independent of $sk_{\{e_1\}}$ by PRF security of KDF (Lemma 5.1 below).

Therefore A 's only path to a recovered sk_e is via direct memory access during the ceremony window — which is excluded by the SMA model. The remaining path is for A to produce a forgery (m', σ') without recovering sk_e , using only (pk_e, σ_token) ; this is exactly a UF-CMA forgery under pk_e with no signing oracle, and succeeds with probability $\leq \text{negl}(\lambda)$ by UF-CMA security of the signing scheme. Both paths are negligible. ■

Lemma 5.1 — KDF Epoch Independence

Let $KDF = HKDF\text{-}SHA3\text{-}512$, modeled as a secure PRF. For any two distinct epoch identifiers $eid_1 \neq eid_2$ and any secret s :

$$KDF(s, \text{salt}=\text{EpochBind}(eid_1, \cdot)) \equiv_c KDF(s, \text{salt}=\text{EpochBind}(eid_2, \cdot))$$

(computationally indistinguishable)

Proof: $\text{EpochBind}(eid, \text{data}) = \text{SHA3-256}(eid \parallel "||" \parallel \text{data})$. For $eid_1 \neq eid_2$, $\text{EpochBind}(eid_1, \cdot) \neq \text{EpochBind}(eid_2, \cdot)$ with overwhelming probability (collision resistance of SHA3-256). HKDF with distinct salts and a PRF-secure hash function produces computationally independent outputs. This follows from the HKDF security analysis (Krawczyk 2010, Theorem 1). ■

5.3 Epoch Isolation (EI)

Definition 5.3: An EKS satisfies **Epoch Isolation** if for all PPT A and all distinct epochs $e_1 \neq e_2$:

$$\Pr[\text{Verify}(\text{pk}_{\{e_2\}}, m^*, A(\text{sk}_{\{e_1\}}, \text{pk}_{\{e_1\}}, \text{pk}_{\{e_2\}}), e_2, t_{\text{now}}) = 1] \leq \text{negl}(\lambda)$$

Proof: Follows directly from Lemma 5.1. Knowledge of $\text{sk}_{\{e_1\}} = KDF(s, \text{salt}_1)$ gives no information about $\text{sk}_{\{e_2\}} = KDF(s, \text{salt}_2)$ for $\text{salt}_1 \neq \text{salt}_2$, by PRF security of HKDF. Therefore A 's advantage in forging under $\text{pk}_{\{e_2\}}$ reduces to its UF-CMA advantage against the signing scheme with an unknown key — $\text{negl}(\lambda)$. ■

5.4 Quantum Forward Secrecy (QFS)

Definition 5.4 — QFS Game:

Game $QFS_A(\lambda)$: $pp \leftarrow \text{Setup}(\lambda)$ $eid \leftarrow \text{EpochGen}(\lambda)$ with $T_{\text{end}} = T_{\text{start}} + T_e$

// Phase 1: Honest ceremony $(\text{sk}_e, \text{pk}_e, W) \leftarrow \text{Assemble}(\dots)$ $\sigma_{\text{token}} \leftarrow \text{Sign}(\text{sk}_e, m, eid)$ $\text{Destroy}(\text{sk}_e)$ // sk_e zeroed; SMA model applies

// Phase 2: Adversary is QPT; epoch has expired (T_e has elapsed) // A sees pk_e on the wire (it's in the token) // A attempts to forge a token for a new claim $m^* \neq m$: $(m, \sigma) \leftarrow A^{\text{QPT}}(\text{pk}_e, \sigma_{\text{token}}, eid, W, pp)$

return $\text{Verify}(\text{pk}_e, m, \sigma, eid, t_{\text{now}}) = 1$ // Note: epoch is expired, so $\text{EpochValid}(eid, t_{\text{now}}) = 0$ // Verify returns 0 regardless of signature validity

Definition 5.5 — Quantum Forward Secrecy: An EKS satisfies QFS if for all QPT adversaries A in the SMA model:

$$\Pr[QFS_A(\lambda) = 1] \leq \text{negl}(\lambda)$$

QFS does not prevent an adversary from eventually computing the private key.

Instead, it ensures that by the time such computation is possible, the key is no longer valid within the protocol.

Security is achieved not by resisting computation indefinitely, but by rendering delayed computation useless.

To be precise: QFS does not guarantee that a valid signature cannot be constructed after epoch expiry. It guarantees that no such signature can ever be accepted by the protocol.

Theorem 5.1 — The Ephemeral-Quantum Gap Theorem

Theorem: Let EKS be an Ethereum Key Scheme with epoch lifetime T_e , satisfying the SMA model (Definition 3.1) and Epoch Isolation (Definition 5.3). Let $T_q(\lambda)$ be the minimum time for any QPT algorithm to solve the hard problem underlying the signing primitive (e.g., ECDLP for Ed25519).

If $T_e < T_q(\lambda)$ is maintained as a deployment invariant, then EKS satisfies QFS.

Proof:

Let A be any QPT adversary in the QFS game. A wins by producing a valid (m, σ) such that $\text{Verify}(\text{pk}_e, m, \sigma, \text{eid}, t_{\text{now}}) = 1$.

Inspect the Verify predicate:

Verify returns 1 iff: (a) signature σ^* is valid under pk_e for message m^* , AND (b) $\text{EpochValid}(\text{eid}, t_{\text{now}}) = 1$, AND (c) eid is not revoked

Since the QFS game runs Phase 2 after the epoch expires (T_e has elapsed), condition (b) is FALSE at the time A attempts forgery. Therefore $\text{Verify} = 0$ unconditionally, regardless of A 's capabilities or the validity of σ^* .

Sub-case: A completes quantum attack before epoch expires ($T_{\text{attack}} < T_e$) A recovers sk_e via the quantum algorithm in time T_{attack} . By assumption $T_e < T_q(\lambda)$, and $T_{\text{attack}} \geq T_q(\lambda)$ for any QPT A (by definition of T_q as the minimum quantum attack time). Therefore $T_{\text{attack}} \geq T_q(\lambda) > T_e$ — contradiction: A cannot complete before T_e . This sub-case is impossible given the invariant.

Sub-case: A forges without recovering sk_e (using only $\text{pk}_e, \sigma_{\text{token}}$) A 's probability of producing a valid σ^* for $m^* \neq m$ is exactly A 's UF-CMA advantage against the signing scheme. Even for a QPT adversary against a classically-vulnerable scheme (Ed25519), this advantage requires either: (i) recovering sk_e (handled above), or (ii) a signing oracle (not available post-ceremony). Therefore this probability $\leq \text{negl}(\lambda)$ under standard UF-CMA assumptions.

Sub-case: A uses recovered $\text{sk}_{\{e\}}$ to forge for a different epoch e_2 By Epoch Isolation (Definition 5.3, proved via Lemma 5.1), A 's advantage reduces to the PRF advantage against HKDF-SHA3-512 — $\text{negl}(\lambda)$.

In all sub-cases, $\Pr[\text{QFS}_A(\lambda) = 1] \leq \text{negl}(\lambda)$. ■

Important qualification: The deployment invariant $T_e < T_q(\lambda)$ is empirical, not mathematical. For Ed25519, with T_q estimated on the order of years (extrapolated from the quantum-resource estimates of Roetteler et al. 2017 — not a direct wall-clock figure; see caveat below) and T_e on the order of hours, the gap is large. But T_q is an estimate with significant uncertainty — and that uncertainty cuts in EKS's favor.

Roetteler et al. give quantum *resource* counts (logical qubits, T-gates), not wall-clock time on future hardware. Converting resource counts to actual run time requires assumptions about:

- Fault-tolerant qubit quality and error correction overhead
- T-gate distillation throughput
- Qubit coherence time at the required circuit depth
- Classical control and measurement latency

Each of these factors inflates T_q beyond the raw circuit estimates. Current physical qubit error rates and coherence times suggest T_q is likely *longer* than Roetteler's optimistic estimates, providing a wider T_e/T_q gap in practice. System operators MUST nevertheless monitor T_q estimates and reduce T_e or migrate to ML-DSA-87 proactively.

QFS under Ed25519 is a time-bounded guarantee, not an unconditional one. Under FES with ML-DSA-87,

this caveat becomes moot — T_q is effectively infinite for ML-DSA-87.

6. Multi-Ceremony Composition

Sequential ceremonies are central to SPECTER's operational model. We address the sequential-composition open problem (Open Problem 1) with a formal conjecture and reduction sketch.

Definition 6.1 — Sequential EKS Composition

A **sequential EKS execution** is a series of ceremonies $C_1, C_2, \dots, C_n$ over disjoint epochs $eid_1, eid_2, \dots, eid_n$ with $T_{end}(eid_i) < T_{start}(eid_{i+1})$. Each ceremony C_i uses the same underlying share set $(\sigma_1, \dots, \sigma_n)$ but a fresh epoch identifier.

Conjecture 6.1 — Sequential Composition Security

Let EKS satisfy EAS (Definition 5.2) and Epoch Isolation (Definition 5.3). Then for any $k \geq 1$ sequential ceremonies, an adversary observing all k public tokens and k public keys gains no advantage in forging a token for epoch eid_{k+1} beyond its UF-CMA advantage against the signing primitive.

Domain Separation Invariant:

Each epoch derives its key material via a PRF with domain separation enforced by EpochBind. Therefore, even under full observation of all prior ceremonies — every public key, every token, every witness record — each epoch key is **computationally independent of all previous keys**. Observation of k ceremonies yields zero information about epoch $k+1$'s key material, by PRF security of HKDF-SHA3-512.

Reduction sketch:

Suppose adversary A , given $(pk_{\{e_1\}}, \sigma_{\{token,1\}}), \dots, (pk_{\{e_k\}}, \sigma_{\{token,k\}})$, can forge a valid token for epoch e_{k+1} .

Construct a reduction R that uses A to break the PRF security of HKDF:

- R is given oracle access to $HKDF(s, \cdot)$ for unknown s
- R simulates k ceremonies, querying the oracle with $salt = \text{EpochBind}(eid_i, aux)$ to obtain $sk_{\{e_i\}}$ for each i
- R gives A the resulting $(pk_{\{e_i\}}, \sigma_{\{token,i\}})$ pairs
- If A outputs a forgery for e_{k+1} , R uses it to distinguish $HKDF(s, salt_{\{k+1\}})$ from random

By PRF security of HKDF-SHA3-512 (Krawczyk 2010), R 's distinguishing advantage is $\text{negl}(\lambda)$. Therefore A 's forgery advantage is also $\text{negl}(\lambda)$.

Status: This is a conjecture. The reduction sketch above is correct in structure but relies on treating HKDF as a PRF even when the adversary has seen k oracle outputs. A full proof requires a hybrid argument over k ceremonies, showing that each additional observed $(pk, token)$ pair adds only $\text{negl}(\lambda)$ information about future sk values. We leave the complete hybrid argument for future work but conjecture it holds under standard PRF assumptions.

7. Full Ethereal Security

Definition 7.1 — Full Ethereal Security (FES)

An EKS achieves **Full Ethereal Security** if:

1. The underlying signature scheme is **QPT-unforgeable** (e.g., ML-DSA-87) — no known quantum attack applies.
2. The EKS satisfies **QFS** (Definition 5.5).
3. The EKS satisfies **EAS** under the SMA model (Definition 5.2).
4. The EKS satisfies **Epoch Isolation** (Definition 5.3).

Theorem 7.1 (*conditional on Conjecture 6.1*): Under FES, no QPT adversary (within the SMA model) can forge a valid token for any epoch, past or future, with probability better than $\text{negl}(\lambda)$.

Proof:

- *Future epochs*: QPT UF-CMA of ML-DSA-87 (the appropriate notion of unforgeability against quantum adversaries; see Boneh & Zhandry 2013) prevents forgery without sk_e . ML-DSA-87 security reduces to the hardness of the Module Learning With Errors (MLWE) problem, for which no QPT algorithm is known (NIST FIPS 204).
- *Current/past epoch forgery post-ceremony*: EAS (Definition 5.2) ensures sk_e is unrecoverable after Destroy under the SMA model.
- *Past epoch forgery using recovered share material*: Epoch Isolation (Definition 5.3) via Lemma 5.1 ensures $\text{negl}(\lambda)$ advantage.
- *Cross-epoch forgery from sequential observations*: **Conditional on Conjecture 6.1** (full proof pending), this is bounded to $\text{negl}(\lambda)$ via PRF security of HKDF. Until Conjecture 6.1 is proved, Theorem 7.1 should be read as: "Under FES, assuming sequential composition security, ..." ■

Remark: Under FES with a QPT-hard signing primitive, the QFS deployment invariant $T_e < T_q(\lambda)$ is no longer necessary for security — ML-DSA-87 provides unconditional resistance to Shor's algorithm, so even if a quantum computer completes in $T_{\text{attack}} < T_e$, it cannot forge under the signing scheme. FES is therefore independent of the T_e/T_q gap.

8. HNDL and the Signature Scheme Distinction

Clarification: Harvest-Now-Decrypt-Later (HNDL) is primarily a threat against *encryption* schemes: an adversary captures ciphertext now and decrypts when quantum hardware is available. For *signature* schemes, there is no ciphertext to decrypt — so the standard HNDL framing is inapplicable.

The relevant threat for signature schemes is subtly different and more interesting:

Token Harvest for Future Forgery: An adversary harvests signed tokens from the wire. The tokens contain pk_e (the Ghost Key public key). When a quantum computer is available, the adversary runs Shor's ECDLP on pk_e to recover sk_e , then uses sk_e to forge new tokens for any identity and scope.

This attack requires:

1. A valid, unexpired epoch at the time of forgery (to pass Verify condition (b))
2. sk_e recovered before the epoch expires

EKS resistance: The epoch expiry check in Verify provides complete resistance: by the time quantum hardware could recover sk_e (T_q on the order of years; see §5.4), the epoch has long since expired. The harvested tokens cannot be used as forgery vehicles because their epoch is revoked.

Under FES (ML-DSA-87): The attack is impossible regardless of epoch timing, because Shor's ECDLP does not apply to ML-DSA-87.

9. SPECTER as an EKS Instantiation

SPECTER (Stateless Protocol for Ephemeral Cryptographic Token Endorsement and Revocation) is the first formally specified instantiation of an EKS.

EKS Component	SPECTER Implementation
Setup	PQCAadapter init, HKDF-SHA3-512 parameters
ShareGen	Shamir (t,n) over Ghost Key seed; shares on USB HRoT
Assemble	Ghost Key ceremony (quorum-gated): Shamir reconstruct → HKDF → ephemeral keypair in SecureBuffer (see §9.4)
Sign	Ed25519 (fallback) or ML-DSA-87 (PQC mode)
Verify	Metatron's Gate: dual-sig check + epoch validity + revocation list
Destroy	SecureBuffer.destroy() — mlock'd buffer, ctypes memset, destructor fallback
Revoke	Epoch revocation in SPECTER audit store (Merkle-chained)

9.1 Liveness in SPECTER

SPECTER satisfies ceremony liveness (Definition 4.3): the deployed LAIRS integration runs SPECTER at its minimum cryptographic threshold $t=2$, with the 3-of-5 quorum enforced one layer above, at the LOCUST council *voting* gate that authorizes the ceremony. A $t=3$, $n=5$ ceremony configuration tolerates two simultaneous share-holder failures (see §9.4 for the deployed-vs-specified threshold reconciliation).

SPECTER satisfies graceful degradation (Definition 4.4): Assemble raises `CeremonyAbortError` on insufficient shares; the SecureBuffer is zeroed in the exception handler before propagation.

Epoch availability (Definition 4.5) depends on council member availability — a protocol liveness concern outside the cryptographic model. The LOCUST dashboard provides real-time availability monitoring.

9.2 Destroy in SPECTER

SPECTER's SecureBuffer (`specter/crypto/memory.py`) allocates a mutable bytearray, pins it in RAM with `mlock(2)` so it cannot be swapped to disk during the ceremony, and zeroizes it with a single compiler-resistant `ctypes.memset` on destroy, with a destructor fallback:

```
import ctypes, ctypes.util
_libc = ctypes.CDLL(ctypes.util.find_library("c"), use_errno=True)
class SecureBuffer:
    def __init__(self, size: int):
        self._size = size
        self._buf = bytearray(size)          # mutable; not the immutable `bytes`
        addr = ctypes.addressof((ctypes.c_char * size).from_buffer(self._buf))
```

```

        self._locked = (_libc.mlock(ctypes.c_void_p(addr),
                                   ctypes.c_size_t(size)) == 0)

        self._destroyed = False
    def destroy(self):
        if self._destroyed:
            return
        addr = ctypes.addressof(
            (ctypes.c_char * self._size).from_buffer(self._buf))
        ctypes.memset(addr, 0, self._size)      # zeroize (not elided by CPython)
        if self._locked:
            _libc.munlock(ctypes.c_void_p(addr), ctypes.c_size_t(self._size))
        self._destroyed = True
    def __del__(self):
        self.destroy()                          # destructor fallback

```

This satisfies Definition 3.2 within the SMA model. It does NOT protect against OS paging or physical memory attacks — those require hardware enclave support (Vettic HRoT integration, planned in LAIRS roadmap).

9.3 SPECTER Security Status

Property	Ed25519 fallback	ML-DSA-87 (PQC)
Classical UF-CMA	YES	YES
QPT UF-CMA	NO (Shor's)	YES (MLWE hardness)
EAS (SMA model)	YES	YES
Epoch Isolation	YES	YES
QFS ($T_e < T_q$ gap)	YES — conditional	YES — unconditional
Sequential composition	Conjecture	Conjecture
Full Ethereum Security	NO	YES

The two columns above are the security endpoints; the deployed backend selects the strongest available at ceremony time, in order: **ML-DSA-87** (liboqs shim — the current live backend) → **ML-DSA-65 / Dilithium-3** (intermediate, via the oqs package) → **Ed25519** (classical dev fallback). Only the ML-DSA-87 path yields Full Ethereum Security.

9.4 Implementation Status and Known Deviations

The formal model (§§3–8) specifies SPECTER's intended design. The deployed build matches it except in the three respects below, which we document explicitly for reproducibility. None affects the QFS, EAS, or Epoch Isolation results.

1. **Key generation.** §5.2 specifies the ephemeral key as a deterministic derivation, $sk_e = \text{KDF}(s, \text{salt} = \text{EpochBind}(\text{eid}))$. The current build instead generates a *fresh* ephemeral keypair inside the ceremony (in the mlock'd SecureBuffer) and destroys it after a single signature; the HKDF-derived seed is computed but not yet threaded into key generation. This is because liboqs 0.15.0 exposes deterministic seeded keygen for KEMs (`OQS_KEM_*_keypair_derand`) but not for ML-DSA signatures. The security results are unaffected: under

fresh-random keys, Epoch Isolation and EAS hold by key independence and non-persistence (a key that is never a function of the shares cannot be recovered from them); under the specified derived keys, they hold via Lemma 5.1. Deterministic seeded keygen — making the build match §5.2 verbatim and cryptographically binding the key to the reconstructed secret — is pending SIG-level support in the PQC backend.

2. **Quorum distribution.** §2.2 and §9.1 describe an (t,n) ceremony in which each principal independently holds a share. The deployed single-host build generates the master secret on the ceremony host and splits/reconstructs it locally, while principals (Hardware Identity Anchors) contribute attestations and an optional ML-KEM seed contribution. Fully distributed share custody — each principal holding its own share — is the intended design and is exercised in the Vettic Sentry-Node bench path; the single-host build *simulates* distribution.
3. **Threshold.** The deployed SPECTER cryptographic threshold is $t=2$ (its minimum). The 3-of-5 governance quorum is enforced at the LOCUST council voting layer that authorizes each ceremony, not inside the SPECTER threshold.

10. Relation to Existing Work

10.1 Forward Secrecy (TLS)

TLS forward secrecy protects session keys: compromise of the long-term server key does not reveal past session keys. However, the long-term server key *persists* and is a permanent attack target. EKS eliminates the long-term asymmetric key entirely. Where TLS provides forward secrecy *despite* a persistent long-term key, EKS provides it *because* no persistent key exists.

Precise distinction: TLS with DHE/ECDHE generates ephemeral session keys per handshake, but the *identity* key (server certificate private key) is long-lived. An adversary who compromises the identity key can impersonate the server in future connections. EKS has no identity key — only epoch-scoped assembled keys that are destroyed immediately. There is no long-lived key to compromise.

10.2 Signal Protocol / PQXDH

Signal's PQXDH (Post-Quantum Extended Diffie-Hellman, 2023) achieves post-quantum forward secrecy for messaging via ephemeral key deletion after each key exchange. This is conceptually close to EKS in motivation.

Precise distinctions:

- PQXDH targets *encryption* key exchange (KEM); EKS targets *signing* and authorization tokens. The threat models are different.
- PQXDH ephemerals are pre-generated and stored (prekey bundles on a server) until consumed. EKS keys are never stored at all — assembled only at the moment of ceremony.
- PQXDH has no threshold assembly or quorum requirement; the ephemeral key is under sole custody of one device. EKS requires t -of- n quorum, making key assembly a governed, auditable event.
- PQXDH's "deleted ephemeral" provides forward secrecy contingent on secure deletion from device storage. EKS's key is never in storage — it exists only in RAM for the duration of the ceremony window.

EKS and PQXDH are complementary: PQXDH addresses the communication layer; EKS addresses the authorization token layer.

10.3 Threshold Signatures (FROST, BLS multisig)

Threshold signature schemes allow n parties to sign without any single party holding the full key. However, the signing *shares* persist across ceremonies and across arbitrary time horizons.

Precise distinction from EKS:

- In FROST/BLS, the long-term signing key is distributed as shares that persist indefinitely. A patient adversary who compromises t shares — even years apart — recovers the signing key.
- In EKS, shares persist (on USB HRoT), but the *assembled key* exists only during the ceremony window. The assembled key is epoch-bound: even if an adversary recovers all shares post-ceremony, they cannot reconstruct a key that is valid in any current or future epoch (Epoch Isolation, Definition 5.3).
- EKS is therefore strictly stronger in the post-ceremony adversary model. The threat surface is the *shares* (mitigated by hardware custody), not the assembled key (which never persists).

10.4 One-Time Signatures (Lamport, SPHINCS+)

One-time signature schemes restrict key use to prevent cryptanalytic attack. EKS is orthogonal: the key is destroyed after one use not to prevent cryptanalysis but to prevent theft. The security arguments are independent and composable. A one-time signature scheme instantiated within EKS would provide both properties simultaneously.

10.5 Hardware Security Modules (HSMs)

HSMs protect persistent keys via tamper-resistant hardware. EKS provides a complementary guarantee: the assembled key does not persistently exist *anywhere*, including HSM memory. The shares (analogous to HSM key material) are protected elements, but they are transformed into an ephemeral key that neither lives in the HSM nor anywhere else. EKS + HSM (for share storage) is a natural composition and is realized in SPECTER via USB HRoT share custody.

Summary of distinctions: The following table characterizes precisely how EKS differs from related schemes along the dimensions that matter most:

Property	TLS-FS	PQXDH	FROST	HSM	EKS
Long-term key persists	YES	YES	YES (shares)	YES	NO
Assembled key stored	N/A	YES (prekey)	NO	YES	NO
Quorum required for signing	NO	NO	YES	NO	YES
Epoch-scoped validity	NO	NO	NO	NO	YES
Post-quantum sign primitive	Partial	YES	Partial	Varies	YES (PQC mode)
Post-ceremony key unrecoverable	NO	Partial	NO	NO	YES (SMA model)

11. Open Problems

1. **Complete hybrid proof for Conjecture 6.1** — Sequential multi-ceremony composition security via full hybrid argument over k ceremonies.
2. **QROM security** — Extend QFS proof to the quantum random oracle model, addressing the use of SHA3-512 in HKDF.
3. **Formal EAS reduction under physical adversary** — Extend the SMA model to include hardware enclave guarantees (SGX, TrustZone) and provide EAS security against physical memory adversaries. The Vettic HRoT integration in LAIRS is the intended realization.
4. **Share compromise model** — Formalize security when up to $t-1$ shares are compromised *before* the ceremony begins. Conjectured to follow from Shamir privacy, but a complete game-based reduction is needed.
5. **Secure erasure as a formal primitive** — Model Destroy as a cryptographic oracle, define its security properties independently of the SMA model, and provide a composability proof. This remains the deepest open problem in the formal treatment.

12. Conclusion

We have defined Ethereal Cryptography as a paradigm in which private key material has no persistent existence — assembled from threshold shares, used once, and destroyed within a formally bounded threat model. The central security property, Quantum Forward Secrecy, captures the intuition that a quantum adversary arriving after a ceremony gains no advantage: the key is gone, the epoch is expired, and cross-epoch attacks are blocked by computationally independent epoch binding.

The Ephemeral-Quantum Gap Theorem formalizes this as a conditional guarantee: if epoch lifetimes are shorter than the quantum attack horizon, QFS holds independently of the signing primitive's quantum hardness. We state this condition explicitly throughout as an empirical deployment invariant, not a mathematical theorem.

Composed with ML-DSA-87, the system achieves Full Ethereal Security — forgery is computationally infeasible for classical and quantum adversaries alike, with security reducing to MLWE hardness and PRF security of HKDF-SHA3-512.

The critical remaining formalization is Destroy as a cryptographic primitive (Open Problem 5) and the complete sequential composition proof (Open Problem 1). SPECTER is the first formally specified system instantiating these principles.

References

- Shamir, A. (1979). How to share a secret. *CACM*, 22(11).
- Shor, P. (1994). Algorithms for quantum computation. *FOCS*.
- Krawczyk, H. (2010). Cryptographic extraction and key derivation: The HKDF scheme. *CRYPTO*.
- Roetteler, M. et al. (2017). Quantum Resource Estimates for Computing Elliptic Curve Discrete Logarithms. *ASIACRYPT*. IACR ePrint 2017/598.
- NIST (2024). FIPS 204: Module-Lattice-Based Digital Signature Standard (ML-DSA).
- NIST (2024). FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism (ML-KEM).
- Bernstein, D.J. & Lange, T. (2017). Post-quantum cryptography. *Nature*, 549.

- Boneh, D. & Zhandry, M. (2013). Secure signatures and chosen ciphertext security in a quantum computing world. *CRYPTO*.
- Nick, J. et al. (2021). FROST: Flexible Round-Optimized Schnorr Threshold Signatures. *SAC*.